Clases abstractas e interfaces

Expliquemos el concepto de clase abstracta y de interfaces (que es una clase totalmente abstracta para implementar herencia multiple) veamos ejemplos en java

Vamos a explicarlo en modo **docente + práctico**, como venís trabajando 👇

🔷 1. Clase Abstracta en Java

Una **clase abstracta** es una clase que:

- **NO se puede instanciar**
- Puede tener:
  - Métodos **abstractos** (sin implementación)
  - Métodos **con implementación**
- Se usa cuando querés definir una **base común** para varias clases

👉 Se declara con `abstract`

📌 Ejemplo básico

Java

```java
abstract class Animal {

    // Método abstracto (sin cuerpo)
    public abstract void hacerSonido();

    // Método normal (con implementación)
    public void dormir() {
        System.out.println("El animal está durmiendo");
    }
}
```

📌 Clase que hereda

Java

```java
class Perro extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Guau!");
    }
}
```

📌 Uso

```java
public class Main {
    public static void main(String[] args) {
        Animal miPerro = new Perro();
        miPerro.hacerSonido();
        miPerro.dormir();
    }
}
```

🔑 Ideas clave

- Sirve para **compartir comportamiento**
- Puede tener **atributos**
- Puede tener **constructores**
- Representa una relación **“es un” (IS-A)**

🔷 2. Interfaces en Java

Una **interfaz** es una “clase totalmente abstracta” (conceptualmente):

- Solo define **qué hacer**, no cómo
- Permite **herencia múltiple**
- Todos los métodos son:
  - `public`
  - `abstract` (por defecto)

👉 Se declara con `interface`

📌 Ejemplo

```java
interface Volador {

    void volar();
}
```

📌 Implementación

```java
class Pajaro implements Volador {

    @Override
    public void volar() {
        System.out.println("El pájaro vuela");
    }
}
```

📌 Uso

```java
public class Main {
    public static void main(String[] args) {
        Volador ave = new Pajaro();
        ave.volar();
    }
}
```

🔷 3. Herencia múltiple con interfaces

Java **NO permite herencia múltiple de clases**, pero sí de interfaces.

📌 Ejemplo

```java
interface Nadador {
    void nadar();
}
```

```java
interface Volador {
    void volar();
}

class Pato implements Nadador, Volador {

    @Override
    public void nadar() {
        System.out.println("El pato nada");
    }

    @Override
    public void volar() {
        System.out.println("El pato vuela");
    }
}
```

📌 Uso

```java
public class Main {
    public static void main(String[] args) {
        Pato pato = new Pato();
        pato.nadar();
        pato.volar();
    }
}
```

🔷 4. Diferencias clave (para examen 👇)

Característica	Clase Abstracta	Interfaz
Instanciable	❌ No	❌ No
Métodos con código	✅ Sí	⚠️ Solo default/static
Métodos abstractos	✅ Sí	✅ Sí
Atributos	✅ Sí	⚠️ Solo constantes
Constructores	✅ Sí	❌ No

Característica	Clase Abstracta	Interfaz
Herencia múltiple	❌ No	✅ Sí

🔷 5. Cuándo usar cada uno

✔️ **Clase abstracta** cuando:

- Compartís lógica común
- Tenés atributos y comportamiento base

✔️ **Interfaz** cuando:

- Definís comportamientos
- Querés **flexibilidad y múltiples roles**

🔷 6. Ejemplo combinado (muy importante)

Java

```java
abstract class Vehiculo {
    String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void mostrarMarca() {
        System.out.println("Marca: " + marca);
    }

    public abstract void moverse();
}
```

Java

```java
interface Electrico {
    void cargarBateria();
}
```

Java

```java
class AutoElectrico extends Vehiculo implements Electrico {

    public AutoElectrico(String marca) {
```

```
        super(marca);
    }

    @Override
    public void moverse() {
        System.out.println("El auto se mueve");
    }

    @Override
    public void cargarBateria() {
        System.out.println("Cargando batería...");
    }
}
```

🧠 RESUMEN FINAL (tipo pizarrón)

- **Clase abstracta → base con lógica**
- **Interfaz → contrato (qué debe hacer)**
- **Interfaces → permiten herencia múltiple**

Si querés, en el próximo paso te armo:
✔️ un **TP para alumnos**
✔️ o un **PPT listo para clase**
✔️ o ejercicios tipo parcial con resolución

¿Seguimos por ahí?

veamos una interface que tenga atributos (de tipo final y static) y que tenga métodos default

Perfecto, vamos a ver un ejemplo **bien completo y claro** de una **interfaz en Java con:**

✔️ atributos `static final`
✔️ métodos `default`
✔️ método abstracto (para comparar)

🔷 1. Concepto clave

En una **interfaz**:

- Los atributos son **SIEMPRE**:

  ```Java
  public static final
  ```

  👉 aunque no lo escribas
- Los métodos pueden ser:
  - `abstract` (por defecto)
  - `default` (con implementación)
  - `static`

🔷 2. Ejemplo completo de interfaz

```Java
interface Dispositivo {

    // Atributos (constantes)
    int VOLTAJE_MAXIMO = 220;     // public static final implícito
    String MARCA = "Genérica";    // constante

    // Método abstracto
    void encender();

    // Método default (con implementación)
    default void mostrarInfo() {
        System.out.println("Marca: " + MARCA);
        System.out.println("Voltaje máximo: " + VOLTAJE_MAXIMO);
    }

    // Método static
    static void mensajeGeneral() {
        System.out.println("Todos los dispositivos deben ser seguros.");
    }
}
```

🔷 3. Clase que implementa la interfaz

```Java
```

```java
class Televisor implements Dispositivo {

    @Override
    public void encender() {
        System.out.println("El televisor se encendió");
    }
}
```

🔷 4. Uso en el main

```java
public class Main {
    public static void main(String[] args) {

        Televisor tv = new Televisor();

        // Método abstracto implementado
        tv.encender();

        // Método default (ya viene implementado)
        tv.mostrarInfo();

        // Método static de la interfaz
        Dispositivo.mensajeGeneral();

        // Acceso a constantes
        System.out.println(Dispositivo.MARCA);
    }
}
```

🔷 5. Salida esperada

```
El televisor se encendió
Marca: Genérica
Voltaje máximo: 220
Todos los dispositivos deben ser seguros.
Genérica
```

🔷 6. Punto clave (muy importante para alumnos ⚠️)

👉 Esto:

```java
int VOLTAJE_MAXIMO = 220;
```

Es lo mismo que:

```java
public static final int VOLTAJE_MAXIMO = 220;
```

🔷 7. Sobre métodos default

Los métodos `default`:

✔️ Tienen implementación
✔️ No obligan a sobrescribir
✔️ Permiten **evolucionar interfaces sin romper código**

📌 Ejemplo de override opcional

```java
class Radio implements Dispositivo {

    @Override
    public void encender() {
        System.out.println("Radio encendida");
    }

    @Override
    public void mostrarInfo() {
        System.out.println("Radio personalizada");
    }
}
```

🧠 RESUMEN TIPO EXAMEN

- Interface = contrato
- Atributos = `public static final`
- Métodos:

- `abstract` → obligatorios
- `default` → opcionales
- `static` → propios de la interfaz

Si querés, en el próximo paso podemos hacer:

✔️ un ejercicio tipo parcial
✔️ o comparar **interface vs clase abstracta con código real lado a lado**
✔️ o armar un caso práctico tipo **home banking (tu proyecto)** usando interfaces 👌

no veo la diferencia entre metodo default y metodo static

Buena pregunta — acá está **la clave conceptual que suele confundir** 👇

🔷 Diferencia central (en una frase)

- `default` → **se usa desde el objeto**
- `static` → **se usa desde la interfaz**

🔷 1. Método `default`

👉 Pertenece a los **objetos de la clase que implementa la interfaz**

✔️ Se **hereda automáticamente**
✔️ Puede ser **sobrescrito (override)**
✔️ Se invoca con el objeto

📌 Ejemplo

Java

```java
interface Dispositivo {

    default void encender() {
        System.out.println("Encendiendo dispositivo...");
```

```
    }
}
```

```java
class Televisor implements Dispositivo {
}
```

```java
public class Main {
    public static void main(String[] args) {
        Televisor tv = new Televisor();

        tv.encender(); // ✅ usa método default
    }
}
```

🔷 2. Método `static`

👉 Pertenece a la **interfaz**, NO al objeto

✔️ **NO se hereda**
✔️ **NO se puede overridear**
✔️ Se invoca con el nombre de la interfaz

📌 Ejemplo

```java
interface Dispositivo {

    static void mensaje() {
        System.out.println("Mensaje general");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        Dispositivo.mensaje(); // ✅ correcto
```

```
        // Televisor tv = new Televisor();
        // tv.mensaje(); ❌ ERROR
    }
}
```

🔷 3. Comparación directa (clave)

Característica	`default`	`static`
Pertenece a	Objeto	Interfaz
Se hereda	✅ Sí	❌ No
Se puede overridear	✅ Sí	❌ No
Cómo se usa	`obj.metodo()`	`Interfaz.metodo()`

🔷 4. Ejemplo combinado (muy claro)

Java

```java
interface Dispositivo {

    default void encender() {
        System.out.println("Encendido básico");
    }

    static void info() {
        System.out.println("Info general");
    }
}
```

Java

```java
class Radio implements Dispositivo {

    @Override
    public void encender() {
        System.out.println("Radio encendida");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        Radio r = new Radio();

        r.encender();        // usa default (o override)
        Dispositivo.info();    // usa static

        // r.info(); ❌ no existe en el objeto
    }
}
```

🔷 5. Analogía (para fijarlo rápido 🧠)

- **default → comportamiento del objeto**

  > "Todos los dispositivos *saben encenderse*"

- **static → utilidad general**

  > "Regla general de dispositivos"

🧠 RESUMEN FINAL (tipo pizarrón)

✔️ `default`
→ vive en el objeto
→ se hereda
→ se puede modificar

✔️ `static`
→ vive en la interfaz
→ no se hereda
→ no se modifica

Si querés, te armo un **ejercicio tipo parcial** donde el alumno tenga que decidir cuándo usar cada uno (ahí se entiende perfecto).